



Bidding Player

Prebid + GAM video header bidding

Publisher Integration Guide

A drop-in JavaScript tag that runs a Prebid header-bidding auction against limelightDigital, stitches the winning bid into your GAM ad call, and plays the resulting VAST creative through video.js + Google IMA.

VERSION

v1.2.2

RELEASED

2026

LICENSE

MIT

REPOSITORY

github.com/aryanvani-pm/bidding-player

ENGINE URL

cdn.jsdelivr.net/gh/aryanvani-pm/bidding-player@v1.2.2/engine/player.js

DOCUMENTATION

aryanvani-pm.github.io/bidding-player/docs.html

Contents

01 Overview

02 How it works

03 Prerequisites

04 Quick start

05 Embed code reference

06 GAM setup

07 Limelight onboarding

08 Consent management

09 Testing & verification

10 Troubleshooting

11 Browser support

12 FAQ

13 Support

01 — Overview

Bidding Player is a drop-in JavaScript tag that lets a publisher monetize their video inventory with real-time header bidding without writing any Prebid integration code.

A single `<script>` tag, configured with `data-*` attributes, handles everything:

- Lazy-loads video.js, Google IMA SDK, and a custom Prebid build (≈ 250 KB total, parallelized).
- Registers an instream video ad unit with the `limelightDigital` bidder.
- Runs the auction against your configured timeout (default 1200 ms).
- Stitches the full Prebid targeting key-value set (`hb_pb` , `hb_bidder` , `hb_uuid` , `hb_size` , `hb_format` , `hb_adid` , plus bidder-suffixed variants) into your GAM master ad tag's `cust_params` .
- Hands the final VAST URL to IMA, which serves either the winning bid's cached creative (resolved via `hb_uuid` against Prebid Cache) or your house ad if no bid wins.

The engine is hosted at versioned, immutable jsDelivr URLs. A tag pinned to `@v1.2.2` serves the exact bytes that were tested at release time and cannot silently change.

WHAT'S BUNDLED

- video.js v8 (player UI)
- Google IMA SDK v3 (ad rendering)
- Prebid.js v11.11.0 with the `limelightDigital` adapter, TCF v2 / USP consent modules

WHAT YOU PROVIDE

- A Limelight publisher ID + ad unit ID
- Your GAM master video ad tag
- GAM line items targeting `hb_*` keys
- The content video URL

02 — How it works

The end-to-end auction-to-impression sequence:

- 1 **Browser** loads the publisher page; the engine's async `<script>` executes.
- 2 **Engine** reads its own `data-*` attributes for runtime configuration.
- 3 **Engine** parallel-loads `video.js`, IMA SDK, and `Prebid.js` from CDN.
- 4 **Engine** measures the mount-div's rendered size; registers an instream video ad unit.
- 5 **Prebid** → **Limelight**: POST OpenRTB request to `{host}/ortbhb`.
- 6 **Limelight** → **Prebid**: bid response (or no-bid).
- 7 **Prebid** caches the winning VAST at `prebid.adnxs.com/pbc/v1/cache`; receives a UUID.
- 8 **Engine** calls `pbjs.getAdserverTargetingForAdUnitCode` to obtain the full `hb_*` key set.
- 9 **Engine** applies its floor bias to `hb_pb`; stitches all keys into GAM's `cust_params`.
- 10 **Engine** → **IMA**: `AdsRequest.adTagUrl = <stitched GAM URL>`.
- 11 **IMA** → **GAM**: fetches VAST; matches your line item targeting `hb_pb`.
- 12 **GAM** returns a VAST wrapper whose URL contains `{hb_uuid}`; GAM substitutes the macro with the cached bid's UUID.
- 13 **IMA** → **Prebid Cache**: resolves the wrapper, retrieves the actual bidder VAST.
- 14 **IMA** plays the ad. **Engine** pauses content (z-index swap on ad container).
- 15 Ad completes; **Engine** swaps z-index back; content resumes.

FALLBACK PATH

If Prebid fails to load, the bidder times out, or no bid comes back, the engine still calls GAM with `hb_pb=0.00&hb_bidder=none` in `cust_params`. GAM's house ad / passback line item fills the impression. The publisher never loses an ad opportunity to a network failure.

03 — Prerequisites

Before integrating, confirm you have all of the following:

REQUIREMENT	WHERE TO GET IT
Google Ad Manager account with video inventory enabled	admanager.google.com
GAM Master Ad Tag URL for the video slot (must end with <code>correlator=</code>)	GAM → Inventory → Ad units → "Generate tags"
Limelight Digital publisher account (publisher ID + ad unit ID)	Limelight account management
Limelight allowlist approval for your domain(s)	Submit via your Limelight account manager. 1–3 business days typical.
HTTPS publisher pages (mixed content blocks IMA + bidder calls)	Your hosting
Content video URL (.mp4 or .m3u8)	Your CDN

ALLOWLIST CHECK

The most common cause of no bid response in production is the publisher's domain not being on Limelight's allowlist. Confirm allowlist status with Limelight *before* launching, otherwise every auction will return no-bid and you'll think the integration is broken.

04 — Quick start

Five steps from zero to a live monetizing player.

1 Generate your tag

Open the Tag Generator at `aryanvani-pm.github.io/bidding-player` and fill in:

- Release version (use the latest tag, e.g. `v1.2.2`)
- Your **GAM Master Ad Tag URL**
- Your **Limelight Publisher ID** and **Ad Unit ID**
- Your content video URL

Click ⚡ **Generate Production Tag** and copy the resulting `<script>` snippet.

2 Set up GAM line items + creative

Create one VAST wrapper creative whose URL is your Prebid Cache + `{hb_uuid}` macro, and one price-bucket line item per \$0.10 CPM step. See section 06 for the exact configuration. (You only need to do this once per ad unit.)

3 Embed the snippet

Paste the generated `<script>` tag into your page at the position where the video player should render. The engine auto-creates the mount `<div>` next to the script.

4 Smoke-test in your browser

Open the page with `?debug=true` appended to the URL. In DevTools → Console you'll see `STEP 1` , `STEP 1.5` , then either **WINNER** or **NO MARKET DEMAND**, then `STEP 6` . In the Network tab, filter for `ortbhb` to see the OpenRTB bid request.

5 Monitor

Once live, monitor in GAM Reports for `hb_bidder=limelightDigital` impressions, eCPM, and fill rate. If you don't see Limelight impressions after 24 hours, see section 10.

05 — Embed code reference

The canonical embed snippet. Every configurable knob is a `data-*` attribute on the `<script>` tag.

```
<script src="https://cdn.jsdelivr.net/gh/aryanvani-pm/bidding-player@v1.2.2/engine/playe
r.js"
    id="adtech-player-core"
    data-pub-id="649658371"
    data-adunit-id="972556929"
    data-tag="https://pubads.g.doubleclick.net/gampad/ads?iu=/...&sz=640x480&output=v
ast&correlator="
    data-timeout="1200"
    data-bias="0.10"
    data-video="https://your-cdn.example.com/content.mp4"
    data-autoplay="true"
    data-muted="true"
    data-fluid="true"
    data-loop="false"
    data-preload="metadata"
    data-vpaid="insecure"
    data-div-id="comparos-video-placement"
    data-host="ads-jbi003.rtba.bidsxchange.com"
    data-cache="https://prebid.adnxs.com/pbc/v1/cache"
    data-prebid-url="https://cdn.jsdelivr.net/gh/aryanvani-pm/bidding-player@v1.2.2/p
rebid/prebid.js"
    async></script>
```

Attribute reference

ATTRIBUTE	REQUIRED	DEFAULT	DESCRIPTION
<code>src</code>	Yes	—	Versioned jsDelivr URL of the engine. Always pin to a tag (<code>@v1.2.2</code>), never <code>@main</code> .
<code>id</code>	Yes	<code>adtech-player-core</code>	Must equal <code>adtech-player-core</code> . The engine uses this ID as a fallback for legacy browsers.
<code>async</code>	Yes	—	Required HTML attribute. The engine boots after the DOM is ready and won't block page rendering.
<code>data-pub-id</code>	Yes	—	Your Limelight publisher ID.
<code>data-adunit-id</code>	Yes	—	Your Limelight ad unit ID (numeric).
<code>data-tag</code>	Yes	—	Your full GAM Master Ad Tag URL. Must end with <code>correlator=</code> (no value) so the engine refreshes it per request.
<code>data-prebid-url</code>	Yes	—	URL of the Prebid.js build. Must include <code>limelightDigitalBidAdapter</code> .
<code>data-host</code>	No	<code>ads-jbi003.rtba.bidsxchange.com</code>	Limelight bidder endpoint host. Only change if Limelight directs you to.
<code>data-video</code>	No	—	Content video URL (.mp4 / .m3u8). If omitted, only the pre-roll plays.
<code>data-timeout</code>	No	<code>1200</code>	Prebid bidder timeout in milliseconds. Industry default for video is 1500–2000.
<code>data-bias</code>	No	<code>0.10</code>	Floor bias in USD added to the winning CPM before bucketing.
<code>data-autoplay</code>	No	<code>true</code>	Browser autoplay policy requires <code>muted=true</code> for unattended autoplay.
<code>data-muted</code>	No	<code>true</code>	Whether the player starts muted. Required if <code>autoplay=true</code> in most browsers.
<code>data-fluid</code>	No	<code>true</code>	<code>true</code> = responsive 16:9. <code>false</code> = fixed 640×480.
<code>data-loop</code>	No	<code>false</code>	Loop the content video after it ends.

ATTRIBUTE	REQUIRED	DEFAULT	DESCRIPTION
<code>data-preload</code>	No	<code>metadata</code>	HTML5 video preload hint: <code>metadata</code> , <code>auto</code> , or <code>none</code> .
<code>data-vpaid</code>	No	<code>insecure</code>	IMA VPAID handling: <code>insecure</code> , <code>enabled</code> , or <code>disabled</code> .
<code>data-div-id</code>	No	<code>comparos-video-placement</code>	ID of the mount div. If no element exists at boot, the engine creates one.
<code>data-cache</code>	No	<code>prebid.adnxs.com/pbc/v1/cache</code>	Prebid Cache server. Switch to your own for higher SLA and volume.

DON'T CHANGE VERSIONS PIECEMEAL

The `src` and `data-prebid-url` attributes must always point at the same version tag. The Tag Generator enforces this; if you hand-edit your snippet, keep them in sync.

06 — GAM setup

This is the side that handles the actual ad serving. It requires careful one-time configuration of **custom key-values**, a **VAST wrapper creative**, and **price-bucket line items**.

Step 1: Create custom key-values

In GAM go to **Inventory → Key-values → New key-value**. Create the following:

KEY NAME	VALUE TYPE	SUGGESTED VALUES
<code>hb_pb</code>	Predefined	<code>0.00</code> , <code>0.10</code> , <code>0.20</code> , ..., <code>20.00</code> (one per \$0.10 step)
<code>hb_bidder</code>	Predefined	<code>limelightDigital</code> , <code>none</code>
<code>hb_uuid</code>	Free-form	(UUIDs are dynamic)
<code>hb_size</code>	Predefined	<code>640x360</code> , <code>1280x720</code> , <code>1920x1080</code>
<code>hb_format</code>	Predefined	<code>video</code>
<code>hb_adid</code>	Free-form	(dynamic)
<code>hb_source</code>	Predefined	<code>client</code> , <code>s2s</code>

Step 2: Create the VAST wrapper creative

The creative is a single VAST URL with a macro that pulls the cache UUID at serve time:

```
https://prebid.adnxs.com/pbc/v1/cache?uuid=%%PATTERN:hb_uuid%%
```

The `%%PATTERN:hb_uuid%%` macro is GAM's syntax for "look up the `hb_uuid` key-value from the ad request and inject its value here."

To create:

1. GAM → **Delivery → Creatives → New creative**
2. Select your video ad unit's advertiser
3. Type: **Video → VAST 3.0 ad tag**
4. Paste the URL above into the **VAST tag URL** field
5. Set duration to **0:00**
6. Save

Step 3: Create price-bucket line items

You need one line item per \$0.10 price bucket from `$0.10` up to your maximum expected CPM (e.g. `$20.00` = 200 line items). The structure is identical for each.

PER LINE ITEM

FIELD	VALUE
Name	<code>Prebid HB \$2.50</code> (or your naming convention)
Inventory size	Video — match your ad unit's size(s)
Type	Price Priority
CPM rate	Match the bucket exactly (e.g. <code>\$2.50</code>)
Goal	None (unlimited) or your impression cap
Targeting → Inventory	Your video ad unit
Targeting → Key-values	<code>hb_pb</code> is one of <code>2.50</code>
Creative	The VAST wrapper from Step 2

Use Prebid's official Line Item Generator tool (docs.prebid.org/tools/line-item-generator.html) to bulk-create all 200 line items via CSV upload — manual creation is impractical at this volume.

EXACT-MATCH VALUES MATTER

Prebid's `hb_pb` values are always strings like `"2.50"` (two decimal places, no currency symbol). Your GAM line item targeting must match that exact string. A line item targeting `"2.5"` will not match `"2.50"` and will never serve.

Step 4: Set up house ad fallback

When no bid wins, the engine sends `hb_pb=0.00&hb_bidder=none` . Create one of:

- A **house line item** targeting `hb_pb=0.00` with a low priority and your fallback creative.
- A **remnant line item** targeting the same key-value pointing at AdSense / a tag-based ad network.
- Leave it blank if you're OK with the slot serving nothing on no-bid (rare; most pubs want to fill 100%).

Step 5: Wrap line items in an Order

GAM requires every line item to live inside an Order. Create a single Order called e.g. **"Prebid Header Bidding"** and assign all 200 price-bucket line items + the fallback to it. This lets you pause / activate the entire integration with one click.

07 — Limelight onboarding

The bidder side is owned by Limelight Digital. Steps you (or your account manager at the platform) follow:

1. **Apply for a Limelight account.** If you don't have one, contact your platform's CSM to request publisher onboarding.
2. **Receive your credentials.** Limelight provides:
 - `publisherId` — numeric, e.g. `649658371` → goes into `data-pub-id`
 - `adUnitId` — numeric, e.g. `972556929` → goes into `data-adunit-id`
 - `host` — bidder endpoint, e.g. `ads-jbi003.rtba.bidsxchange.com` → goes into `data-host`
3. **Submit your domain(s) for allowlist.** Limelight only bids on requests from approved publisher domains. Without this step, every auction returns no-bid in production.
4. **Wait for allowlist approval.** Typically 1–3 business days.
5. **Verify bidding is live** using the steps in section 09.

TIP

You can test the *integration mechanics* (script loads, auction fires, GAM call goes out, ad plays from GAM's test creative) before your domain is allowlisted. The bidder side will return no-bid until allowlisting completes — that's why the engine has a fallback path.

08 — Consent management

The engine ships consent-aware defaults appropriate for **non-EU traffic** but requires an override for EU / California / other-state-privacy traffic.

What the engine does by default

FRAMEWORK	DEFAULT BEHAVIOR
GDPR (TCF v2)	Declares <code>gdprApplies: false</code> via static config. The bundled <code>tcfControl</code> module skips enforcement. Suitable for US/India/non-EU traffic.
USP (CCPA)	Auto-detects <code>window.__uspapi</code> with a 100 ms timeout. If no CMP responds, proceeds without USP signal.
GPP (US state privacy)	Not activated. The bundled GPP module cancels the auction outright when no GPP CMP is found, so it is intentionally left off.

EU publishers — enable real TCF v2 consent

If you serve EU traffic, place a registered TCF v2 CMP on your page (OneTrust, Sourcepoint, Quantcast, Didomi, etc.) and override the engine's static GDPR config. Place this snippet *before* the engine's

`<script>` tag:

```
<script>
window.pbjs = window.pbjs || { que: [] };
pbjs.que.push(function() {
  pbjs.setConfig({
    consentManagement: {
      gdpr: {
        cmpApi: 'iab',           // detect window.__tcfapi
        timeout: 8000,           // wait up to 8s for CMP response
        defaultGdprScope: true // assume GDPR applies unless CMP says otherwise
      }
    }
  });
});
</script>
```

California / Connecticut / Virginia (USP)

The engine auto-detects USP. Ensure your CMP exposes `window.__uspapi`. To extend the timeout:

```
pbjs.setConfig({
  consentManagement: {
    usp: { cmpApi: 'iab', timeout: 3000 }
  }
});
```

Multi-state US (GPP)

If your CMP supports GPP, opt in explicitly:

```
pbjs.setConfig({
  consentManagement: {
    gpp: { cmpApi: 'iab', timeout: 3000 }
  }
});
```

WITHOUT A CMP IN REGULATED JURISDICTIONS

Operating in the EU, California, or other regulated states without a registered CMP is a regulatory risk independent of this player. The engine's permissive defaults are a publisher-experience choice — they do **not** constitute legal compliance. Consult counsel.

09 — Testing & verification

Before declaring the integration live, verify each of the following.

Enable debug mode

Append `?debug=true` to your page URL. The engine logs the auction flow in the browser console with color-coded steps.

Console checklist

Open DevTools → Console. Filter for `STEP`. You should see:

- `STEP 0.5` Mount point auto-created (only if you didn't pre-place the div)
- `STEP 1` Auction Initialized via CDN. Target: limelightDigital | Timeout: 1200ms
- `STEP 1.5` Requesting video ad payload. Player size: XXX×YYY
- Either `WINNER` <bidder> | Raw CPM: \$X.XX (real bid) or `NO MARKET DEMAND`
- `STEP 3` Math Engine processing (only on winning bid)
- `STEP 6` Dispatching VAST Request via Cloud Engine layer

Network checklist

Open DevTools → Network. Verify each of these requests fires:

REQUEST	STATUS	WHAT TO CONFIRM
<code>cdn.jsdelivr.net/.../engine/player.js</code>	200	Engine loaded
<code>cdn.jsdelivr.net/.../prebid/prebid.js</code>	200	Prebid bundle loaded
<code>imasdk.googleapis.com/.../ima3.js</code>	200	IMA SDK loaded
<code>POST {host}/ortbhb</code>	200	OpenRTB bid request fired. Inspect Payload tab.
<code>prebid.adnxs.com/pbc/v1/cache</code>	200	Caches the winning VAST (only on winning bid)
<code>GET pubads.g.doubleclick.net/gampad/ads</code>	200	GAM ad call. <code>cust_params</code> must contain <code>hb_pb</code> , <code>hb_bidder</code> , and <code>hb_uuid</code> .

Visual checklist

- The video player renders at the expected size and position on the page.
- A pre-roll ad plays before content.
- Content video plays after the ad completes.
- Standard Video.js controls (play/pause, mute, volume, progress, fullscreen) appear and respond.

10 — Troubleshooting

No `POST /ortbhb` request in the Network tab

Means Prebid never fired the bidder. Possible causes:

- **Adapter not in Prebid build.** Inspect `pbjs.installedModules` in the console — `limelightDigitalBidAdapter` must be present.
- **Consent module blocking the auction.** Look for `"Canceling auction as per consentManagement config"` in console.
- **Adapter rejected the ad unit.** Look for `auctionDebug` messages: *"video ad unit containing bidders that don't support video"* means your Prebid build is missing the adapter.

`POST /ortbhb` returns 200 but bid response is empty

Bidder responded but didn't bid. Almost always:

- **Domain not allowlisted by Limelight.** Verify with Limelight that your origin is on the approved list.
- **Floor below Limelight's threshold.** If you've set a floor, lower it.
- **Ad slot size not supported.** If the player is unusually small (< 300×200), bidders may not bid.

Bid wins but no ad plays / wrong ad plays / GAM falls through to house

The auction worked but GAM failed to resolve the bidder's creative. Check:

- **The GAM URL `cust_params` must contain `hb_uuid`.** If missing, you're on an old engine version (< v1.2.0) — upgrade.
- **Your line item targeting must match `hb_pb` exactly.** Prebid emits `"2.50"`, your line item must target `"2.50"`, not `"2.5"`.
- **Your VAST wrapper creative must use `%%PATTERN:hb_uuid%%`.** Confirm in GAM → Delivery → Creatives.
- **Your Prebid Cache must be reachable** from the user's browser.

`Request timeout after 1200ms` in console

Bidder didn't respond in time. Options:

- Increase `data-timeout` to `1500` or `2000` ms.
- Check Limelight's regional endpoint — ask if they have a closer host.

Ad plays but content doesn't resume after

IMA's `CONTENT_RESUME_REQUESTED` event didn't fire or was caught by a publisher CSS rule. Check that the mount div has no parent rule forcing `pointer-events: none` or `opacity: 0` on the ad container.

Player doesn't autoplay

Browser autoplay policies require `muted=true` for unattended autoplay. Confirm:

- `data-autoplay="true"` AND `data-muted="true"` are both set.
- The page is served over HTTPS.
- The user hasn't disabled autoplay globally in their browser settings.

On iOS Safari, additional restrictions apply — there is no workaround at the engine level.

Console error: `currentScript missing anchor validation`

The engine couldn't locate its own `<script>` tag. Ensure the snippet has `id="adtech-player-core"`. Regenerate the tag if necessary.

Mixed content warnings in the console

Your page is HTTPS but one of these URLs is `http://`: `data-video`, `data-tag`, or a creative returned by Limelight/GAM. The browser will block these. Force all URLs to HTTPS.

11 — Browser support

BROWSER	STATUS	NOTES
Chrome (desktop)	Tested	Reference platform. All features verified.
Edge (desktop, Chromium)	Tested	Equivalent to Chrome.
Firefox (desktop)	Should work	Prebid + IMA + video.js all support Firefox. Test before mass rollout.
Safari (desktop)	Should work	Watch for ITP / 3rd-party cookie restrictions.
iOS Safari (mobile)	Test required	Autoplay restrictions stricter. <code>playsinline</code> set by engine. Prefer SIMID over VPAID.
Android Chrome (mobile)	Test required	Generally well-supported. Android in-app webviews restricted.
IE11 and earlier	Not supported	Engine uses ES2017 features. No IE support.

Minimum browser requirements

- ES2017+ (async/await, modern `URL` API)
- `HTMLMediaElement` (HTML5 video)
- `fetch` + `Promise`
- HTTPS context (no `http://` embed)

12 — FAQ

Can I run multiple players on the same page?

Not in the current engine. The integration assumes a single video ad unit per page. Workaround: load the engine in an iframe per player.

Can I add other Prebid bidders besides limelightDigital?

You need a custom Prebid build that includes the additional adapters. The engine currently registers only one bidder per auction. Adding more bidders requires an engine code change — talk to your account manager.

What happens if Prebid fails to load?

The engine catches the failure and calls GAM directly with `hb_pb=0.00&hb_bidder=none`. Your house ad / fallback line item fills the impression.

How do I track auction performance?

Impression-level metrics (eCPM, revenue, fill rate) are available natively in GAM Reports — filter by the `hb_bidder` key-value. For auction-level metrics (bid latency, bid rate), request a Prebid build with an analytics adapter.

Will ad blockers affect this?

Yes, like all Prebid integrations. Common ad blockers will block the engine script, `prebid.js`, the bidder endpoint, the GAM call, IMA SDK, and the resulting ad. The engine has no special evasion behavior.

What's the latency impact on my page?

The engine script is `async` and doesn't block page rendering. The auction adds ~1200 ms (the configured timeout) before the first ad starts playing. Zero impact on Time to Interactive; a configurable delay only on the pre-roll ad start.

Can I customize the player controls or skin?

The player is `video.js v8` with the default skin. Override `video.js` CSS by placing rules *after* the engine's `<script>` tag, targeting `.video-js` selectors.

How often does the player refresh the auction?

Once per page load. There is no automatic refresh. Mid-roll ads are not currently supported.

What's the difference between `@v1.2.2` and `@main` ?

`@v1.2.2` is an immutable tag — the URL is cached aggressively (1-year CDN TTL) and serves the exact bytes from release time. `@main` points at the latest commit, cached ~12 hours. Use `@main` for development; always pin to a version tag for production.

Is this GDPR / CCPA compliant out of the box?

The engine includes the technical hooks for compliance. Whether your overall integration is compliant depends on placing a registered CMP on your page in regulated jurisdictions, your privacy policy disclosures, and your data processing agreements with bidders. Consult counsel.

13 — Support

For integration help, bug reports, and feature requests:

- **Repository & issues:** github.com/aryanvani-pm/bidding-player
- **Tag generator:** aryanvani-pm.github.io/bidding-player
- **Online documentation:** aryanvani-pm.github.io/bidding-player/docs.html
- **Demo publisher page:** aryanvani-pm.github.io/bidding-player/demo/publisher-test.html?debug=true

Bidding Player • v1.2.2 • Built on Prebid.js, video.js, Google IMA SDK

Engine: cdn.jsdelivr.net/gh/aryanvani-pm/bidding-player@v1.2.2/engine/player.js